Informe Técnico: Agenda de Contactos

Manejo de Archivos con Registros Fijos, Acceso Aleatorio e Indexación Primaria

Facultad de Ingeniería - Campus San Alberto Hurtado, S.J. de Quetzaltenango

1. Estructura del Registro de Tamaño Fijo (256 Bytes)

Para posibilitar el acceso aleatorio directo O(1), cada registro en 'contactos.dat' ocupa exactamente 256 bytes en disco. Se definieron los siguientes campos binarios en UTF-8:

Campo	Bytes	Descripción y Estrategia de Relleno
id	10 bytes	Entero positivo alineado a la derecha con espacios (0x20)
nombre	80 bytes	Nombre completo, codificado en UTF-8, ljust(80, b' ')
telefono	20 bytes	Número telefónico en UTF-8, ljust(20, b' ')
correo	80 bytes	Dirección de correo electrónico, ljust(80, b' ')
ciudad	50 bytes	Ciudad de residencia, ljust(50, b' ')
estado	1 byte	'A'=Activo, 'D'=Eliminado lógicamente
padding	14 bytes	Espacios neutros para alineación a 256 bytes
newline	1 byte	Salto de línea '\n' para inspección limpia en disco
TOTAL	**256 BYTES**	**Longitud estricta por registro físico**

2. Implementación de Acceso Aleatorio con seek() y tell()

El acceso aleatorio ubica la aguja del archivo en: offset = numero_registro * 256 bytes. No se lee el archivo completo. Con tell() se demuestra la posición antes y después de cada I/O:

```
  # Lectura con seek() y tell():
  offset = record_num * 256
  with open('contactos.dat', 'rb') as f:
      f.seek(offset)               # Salto directo a la posición
      pos_bytes = f.tell()         # Demostración tell(): byte exacto
      data = f.read(256)           # Lectura de tamaño fijo

  # Modificación in-place:
  with open('contactos.dat', 'r+b') as f:
      f.seek(offset)               # Salto directo al registro
      f.write(data_256_bytes)      # Sobrescribe SOLO este registro
```

3. Índice Primario JSON y Ventaja Computacional

El índice ('indice.json') mantiene el mapeo: ID -> Número de Registro.

* Búsqueda Secuencial: Complejidad O(N). Para N contactos, requiere leer en promedio N/2 registros de disco.
* Búsqueda con Índice: Complejidad O(1). Consulta en RAM la posición y hace un único seek() directo.
* Si 'indice.json' se corrompe, el sistema incluye la opción 'Reconstruir Índice' que regenera el archivo escaneando los IDs en pasos de 256 bytes sin perder ningún contacto.

4. Integridad Criptográfica (SHA-256) y Respaldo ZIP

* Verificación SHA-256: Cada guardado calcula el hash criptográfico del archivo de datos. Al iniciar el programa, se recalcula y compara. Si se detecta un byte alterado fuera de la aplicación, se notifica inmediatamente al usuario con alerta visual.
* Respaldo Comprimido ZIP: Empaqueta contactos.dat, indice.json y contactos.sha256 usando zipfile (DEFLATE).

Métrica de Almacenamiento	Valor	Observaciones
Tamaño Original sin comprimir	3,297 bytes (3.22 KB)	12 registros × 256B + índice + hash

Tamaño Comprimido en ZIP	1,252 bytes (1.22 KB)	Algoritmo DEFLATE estándar
Espacio Ahorrado	2,045 bytes	Reducción significativa
Porcentaje de Compresión	62.0 % de reducción	Óptimo por relleno uniforme de espacios

5. Interfaz Gráfica Liquid Glass

La interfaz fue desarrollada con Tkinter y Pillow implementando un fondo animado de cristal líquido (Liquid Glass) con blobs de colores gaussianos en movimiento continuo. Cuenta con panel de estadísticas, buscador en tiempo real, tabla con selección y scrollbars, diálogo de metadatos (os.stat/os.access) y barra de estado que refleja la posición actual del cursor de archivo (tell).